LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 14 Security, Exception, Logging, PRG, dan Integrasi CRUD Lengkap

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi web tidak hanya membutuhkan fungsi untuk menampilkan dan mengolah data, tetapi juga memerlukan keamanan input, penanganan kesalahan, pencatatan aktivitas, dan pengaturan alur request. Hal tersebut diperlukan agar aplikasi dapat menangani input pengguna dengan benar dan memberikan respons yang sesuai ketika terjadi kesalahan.

Pada praktikum Acara 14, materi yang dipelajari adalah Security, Exception, Logging, Post/Redirect/Get (PRG), dan Integrasi CRUD Lengkap. Implementasi materi diterapkan pada aplikasi Manajemen Data Mahasiswa berbasis PHP yang menggunakan pemrograman berorientasi objek dan database MySQL.

Aplikasi yang digunakan memiliki pemisahan beberapa komponen, yaitu Controller, Service, Repository, Model, serta komponen pendukung untuk koneksi database, session, flash message, dan logging. Pemisahan ini bertujuan agar pengolahan data dan penanganan kesalahan lebih terstruktur.

1.2 Rumusan Masalah

1. Bagaimana menerapkan keamanan input dan output pada aplikasi web?
2. Bagaimana menangani exception ketika terjadi kesalahan pada aplikasi?
3. Bagaimana mencatat kesalahan aplikasi menggunakan logging?
4. Bagaimana menerapkan pola Post/Redirect/Get pada operasi pengolahan data?
5. Bagaimana mengintegrasikan proses tambah, tampil, ubah, dan hapus data mahasiswa?

BAB 2. LANDASAN TEORI

2.1 Security pada Aplikasi Web

Security merupakan upaya untuk melindungi aplikasi web dan database dari kesalahan input maupun tindakan yang tidak diinginkan. Salah satu penerapannya adalah melakukan validasi terhadap data yang dimasukkan pengguna sebelum data diproses atau disimpan.

2.2 Validasi Input

Validasi input adalah proses memeriksa data yang dimasukkan pengguna agar sesuai dengan ketentuan. Contohnya, NIM tidak boleh kosong, nama wajib diisi, dan email harus memiliki format yang benar.

2.3 Prepared Statement

Prepared statement merupakan metode untuk menjalankan query SQL dengan memisahkan perintah SQL dari nilai input pengguna. Metode ini membantu mencegah serangan SQL Injection dan dapat diterapkan menggunakan PDO pada PHP.

2.4 Exception Handling

Exception handling digunakan untuk menangani kesalahan yang terjadi ketika program dijalankan. Dalam PHP, mekanisme ini dapat menggunakan `try`, `catch`, dan `throw` sehingga aplikasi dapat memberikan respons yang sesuai ketika terjadi kesalahan.

2.5 Logging

Logging adalah proses mencatat kejadian penting atau kesalahan yang terjadi pada aplikasi. Catatan kesalahan dapat disimpan dalam file log agar pengembang lebih mudah mengetahui penyebab masalah tanpa menampilkan informasi teknis kepada pengguna.

2.6 Session dan Flash Message

Session digunakan untuk menyimpan informasi sementara selama sesi pengguna berlangsung. Flash message merupakan pesan sementara yang ditampilkan setelah suatu proses dilakukan, misalnya ketika data mahasiswa berhasil ditambahkan, diubah, atau dihapus.

2.7 Post/Redirect/Get (PRG)

Post/Redirect/Get merupakan pola alur aplikasi yang menjalankan proses POST, kemudian mengarahkan pengguna ke halaman GET. Pola ini membantu mencegah pengiriman ulang data ketika pengguna melakukan refresh setelah mengirimkan formulir.

2.8 CRUD

CRUD merupakan empat operasi dasar dalam pengelolaan data, yaitu Create untuk menambahkan data, Read untuk menampilkan data, Update untuk mengubah data, dan Delete untuk menghapus data. Pada praktikum ini, CRUD diterapkan pada data mahasiswa yang tersimpan dalam database MySQL.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

A. Database penambahan kolom email

```sql
ALTER TABLE mahasiswa
ADD COLUMN email VARCHAR(15) NULL AFTER nama;
```

B. Database penambahan kolom angkatan

Run SQL query/queries on table si_akademik7.mahasiswa:

```sql
ALTER TABLE mahasiswa
ADD COLUMN angkatan YEAR NULL AFTER email;
```

C. Kode Logger.php

```php
<?php

class Logger
{
    public static function path(): string
    {
        return dirname(__DIR__, 2) . '/storage/logs/app.log';
    }

    public static function error(string $message): void
    {
        self::write('ERROR', $message);
    }

    public static function exception(Throwable $e, string
$context = ''): void
    {
        $file = str_replace(
            [dirname(__DIR__, 2) . DIRECTORY_SEPARATOR,
dirname(__DIR__, 2) . '/'],
            '',
            $e->getFile()
        );

        $message = ($context !== '' ? $context . ' - ' : '')
            . $e->getMessage()
            . ' (' . get_class($e) . ' di ' . $file . ':' .
$e->getLine() . ')';
```

```
        self::write('ERROR', $message);
    }

    private static function write(string $level, string
$message): void
    {
        $path = self::path();
        $dir  = dirname($path);

        if (!is_dir($dir)) {
            @mkdir($dir, 0755, true);
        }

        $message = preg_replace('/\s+/', ' ', $message);

        $line = date('Y-m-d H:i:s') . " [$level] " . $message .
PHP_EOL;

        @error_log($line, 3, $path);
    }
}
```

D. Kode MahasiswaService.php

```
<?php

require_once __DIR__ . '/../Core/Logger.php';
require_once __DIR__ . '/../Core/DataException.php';
require_once __DIR__ . '/../Models/MahasiswaRepository.php';
require_once __DIR__ . '/../Models/ProdiRepository.php';


class MahasiswaService
{
    private const STATUS_VALID  = ['aktif', 'lulus', 'cuti'];
    private const ANGKATAN_MIN  = 2000;

    public function __construct(
        private MahasiswaRepository $repo,
        private ProdiRepository $prodiRepo
    ) {}

    public function all(): array
    {
        return $this->read(
            fn() => $this->repo->all(),
```

```
            'MahasiswaService::all',
            'Data mahasiswa gagal dimuat'
        );
    }

    public function find(int $id)
    {
        return $this->read(
            fn() => $this->repo->find($id),
            'MahasiswaService::find',
            'Data mahasiswa gagal dimuat'
        );
    }

    public function findDetail(int $id)
    {
        return $this->read(
            fn() => $this->repo->findDetail($id),
            'MahasiswaService::findDetail',
            'Data mahasiswa gagal dimuat'
        );
    }

    public function prodiOptions(): array
    {
        return $this->read(
            fn() => $this->prodiRepo->names(),
            'MahasiswaService::prodiOptions',
            'Daftar program studi gagal dimuat'
        );
    }

    public function create(array $input): array
    {
        try {
            $input  = $this->normalize($input);
            $errors = $this->validate($input);

            if (!empty($errors)) {
                return $this->fail($errors);
            }

            $id = $this->repo->create($input);

            return [
                'success' => true,
                'id'      => $id,
```

```
                'message' => 'Data mahasiswa berhasil
ditambahkan',
            ];
        } catch (Exception $e) {
            return $this->failFromException($e,
'MahasiswaService::create', 'Data gagal disimpan', true);
        }
    }

    public function update(int $id, array $input): array
    {
        try {
            if ($id <= 0 || !$this->repo->find($id)) {
                return $this->fail(['_global' => 'Data
mahasiswa tidak ditemukan']);
            }

            $input  = $this->normalize($input);
            $errors = $this->validate($input, $id);

            if (!empty($errors)) {
                return $this->fail($errors);
            }

            $this->repo->update($id, $input);

            return [
                'success' => true,
                'id'      => $id,
                'message' => 'Data mahasiswa berhasil diubah',
            ];
        } catch (Exception $e) {
            return $this->failFromException($e,
'MahasiswaService::update', 'Data gagal disimpan', true);
        }
    }

    public function delete(int $id): array
    {
        try {
            if ($id <= 0 || !$this->repo->find($id)) {
                return $this->fail(['_global' => 'Data
mahasiswa tidak ditemukan']);
            }

            $this->repo->delete($id);
```

```
            return [
                'success' => true,
                'message' => 'Data mahasiswa berhasil dihapus',
            ];
        } catch (Exception $e) {
            return $this->failFromException($e,
'MahasiswaService::delete', 'Data gagal dihapus');
        }
    }

    private function validate(array $input, ?int $exceptId =
null): array
    {
        $errors = [];

        // NIM
        if ($input['nim'] === '') {
            $errors['nim'] = 'NIM wajib diisi';
        } elseif (!preg_match('/^[0-9]+$/', $input['nim'])) {
            $errors['nim'] = 'NIM harus berupa angka';
        } elseif (strlen($input['nim']) > 20) {
            $errors['nim'] = 'NIM maksimal 20 karakter';
        } elseif ($this->repo->existsByNim($input['nim'],
$exceptId)) {
            $errors['nim'] = 'NIM sudah terdaftar';
        }

        // Nama
        if ($input['nama'] === '') {
            $errors['nama'] = 'Nama wajib diisi';
        } elseif (strlen($input['nama']) > 100) {
            $errors['nama'] = 'Nama maksimal 100 karakter';
        }

        // Email
        if ($input['email'] === '') {
            $errors['email'] = 'Email wajib diisi';
        } elseif (strlen($input['email']) > 100) {
            $errors['email'] = 'Email maksimal 100 karakter';
        } elseif (!filter_var($input['email'],
FILTER_VALIDATE_EMAIL)) {
            $errors['email'] = 'Format email tidak valid';
        }

        $maxTahun = (int) date('Y');

        if ($input['angkatan'] === '') {
```

```
            $errors['angkatan'] = 'Angkatan wajib diisi';
        } elseif (!preg_match('/^\d{4}$/', $input['angkatan'])
            || (int) $input['angkatan'] < self::ANGKATAN_MIN
            || (int) $input['angkatan'] > $maxTahun) {
            $errors['angkatan'] = 'Angkatan harus berupa tahun
' . self::ANGKATAN_MIN . '-' . $maxTahun;
        }

        // Prodi
        if ($input['prodi'] === '') {
            $errors['prodi'] = 'Program studi wajib dipilih';
        } elseif
(!$this->prodiRepo->existsByNama($input['prodi'])) {
            $errors['prodi'] = 'Program studi tidak tersedia';
        }

        // Status
        if ($input['status'] === '') {
            $errors['status'] = 'Status wajib dipilih';
        } elseif (!in_array(strtolower($input['status']),
self::STATUS_VALID, true)) {
            $errors['status'] = 'Status tidak valid';
        }

        return $errors;
    }

    private function normalize(array $input): array
    {
        return [
            'nim'      => trim((string) ($input['nim'] ?? '')),
            'nama'     => trim((string) ($input['nama'] ??
'')),
            'email'    => trim((string) ($input['email'] ??
'')),
            'angkatan' => trim((string) ($input['angkatan'] ??
'')),
            'prodi'    => trim((string) ($input['prodi'] ??
'')),
            'status'   => trim((string) ($input['status'] ??
'')),
        ];
    }

    private function read(callable $fn, string $context, string
$userMessage)
    {
```

```
        try {
            return $fn();
        } catch (PDOException $e) {
            Logger::exception($e, $context);

            throw new DataException($userMessage, 0, $e);
        }
    }

    private function failFromException(
        Exception $e,
        string $context,
        string $userMessage,
        bool $checkDuplicateNim = false
    ): array {
        Logger::exception($e, $context);

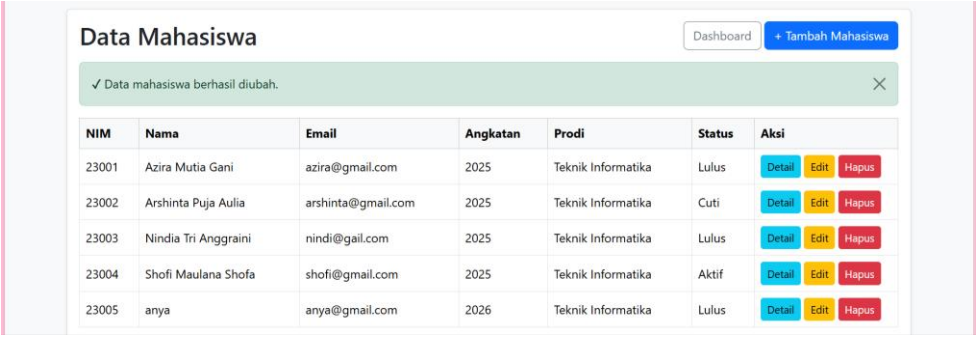
        if ($checkDuplicateNim && $e instanceof PDOException &&
$e->getCode() === '23000') {
            return $this->fail(['nim' => 'NIM sudah
terdaftar']);
        }

        return $this->fail(['_global' => $userMessage]);
    }

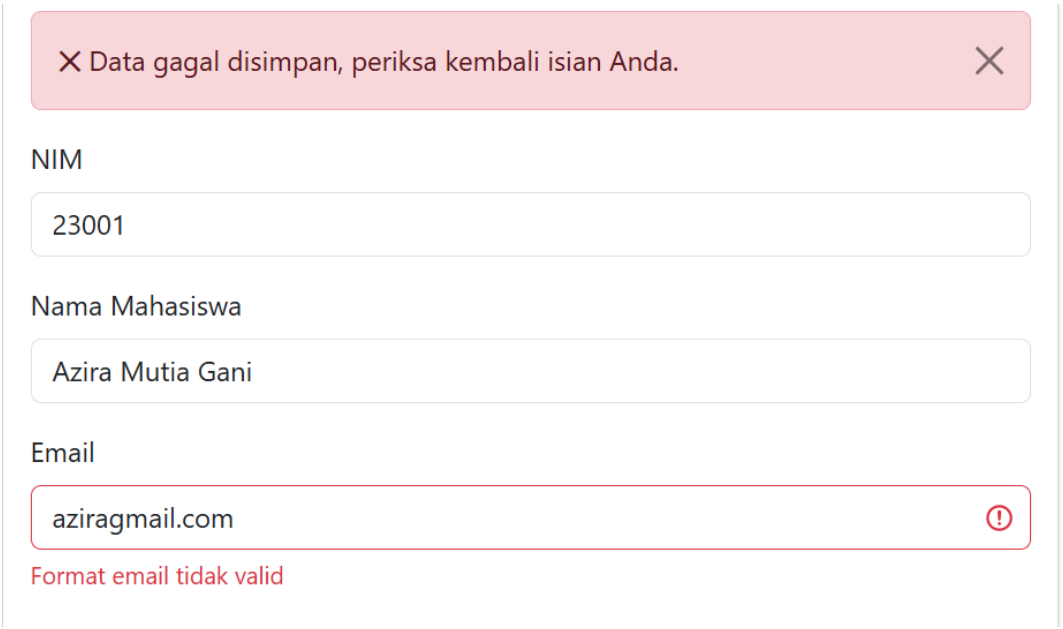
    private function fail(array $errors): array
    {
        return [
            'success' => false,
            'errors'  => $errors,
        ];
    }
}
```

E. Hasil

a) Tampilan data mahasiswa



b) Validasi email yang tidak valid



BAB 4. PEMBAHASAN

4.1 Pembahasan

Pada praktikum Acara 14, aplikasi Sistem Informasi Akademik disempurnakan dengan menerapkan keamanan, penanganan kesalahan, pencatatan log, dan integrasi CRUD. Penerapan validasi membantu memastikan bahwa data mahasiswa yang diproses sesuai dengan ketentuan. Prepared statement digunakan untuk meningkatkan keamanan query database dari input pengguna yang berpotensi berbahaya.

Exception handling membantu aplikasi menangani kesalahan tanpa harus menampilkan detail teknis kepada pengguna. Sementara itu, logging memudahkan pengembang melakukan pemeriksaan apabila terjadi masalah pada aplikasi. Session dan flash message digunakan untuk memberikan informasi mengenai hasil proses yang dilakukan pengguna.

Penerapan PRG juga penting karena setelah proses penambahan, perubahan, atau penghapusan data, pengguna diarahkan kembali ke halaman GET. Dengan demikian, risiko pengiriman ulang formulir akibat refresh halaman dapat dikurangi.

Integrasi CRUD memungkinkan pengelolaan data mahasiswa dilakukan secara lengkap melalui aplikasi. Seluruh fitur tersebut mendukung pengembangan aplikasi yang lebih aman, terstruktur, dan mudah dipelihara.

2.1 Analisis Pengujian

Pengujian dilakukan berdasarkan beberapa kondisi, seperti input valid, input tidak lengkap, NIM duplikat, email tidak valid, perubahan data, penghapusan data, serta kondisi ketika database mengalami kesalahan. Setiap pengujian digunakan untuk memeriksa kesesuaian perilaku aplikasi dengan hasil yang diharapkan.

Apabila ditemukan fitur yang belum berjalan sesuai harapan, perbaikan dapat dilakukan pada bagian Controller, Service, Repository, atau konfigurasi database sesuai sumber masalahnya. Pemeriksaan file log dapat membantu menemukan penyebab kesalahan secara lebih terarah.

BAB 5. PENUTUP

5.1 Kesimpulan

Praktikum Acara 14 membahas penerapan keamanan dan pengelolaan kesalahan pada aplikasi web menggunakan PHP dan MySQL. Materi yang diterapkan meliputi validasi input, prepared statement, exception handling, logging, session, flash message, pola PRG, dan integrasi CRUD.

Penerapan konsep tersebut bertujuan meningkatkan keamanan pengolahan data, mempermudah proses pemeriksaan kesalahan, memberikan informasi kepada pengguna, serta mengurangi risiko pengiriman ulang formulir. Integrasi CRUD juga memungkinkan data mahasiswa dikelola melalui aplikasi secara lebih terstruktur.